

OOP stands for **Object-Oriented Programming**.

Python is an object-oriented language, allowing you to structure your code using classes and objects for better organization and reusability.

Advantages of OOP

- Provides a clear structure to programs
- Makes code easier to maintain, reuse, and debug
- Helps keep your code DRY (**Don't Repeat Yourself**)
- Allows you to build reusable applications with less code

Tip: The DRY principle means you should avoid writing the same code more than once. Move repeated code into functions or classes and reuse it.

What are Classes and Objects?

Classes and objects are the two core concepts in object-oriented programming.

A class defines what an object should look like, and an object is created based on that class.

For example:

Class	Object
Fruit	Apple, Banana, Mango
Car	Volvo, Audi, Toyota

When you create an object from a class, it inherits all the variables and functions defined inside that class.

Python Classes/Objects

Python is an object-oriented programming language.

Almost everything in Python is an object, with its properties and methods.

A Class is like an object constructor, or a "**blueprint**" for creating objects.

Create a Class

To create a class, use the keyword **class**:

Example:

Create a class named MyClass, with a property named x:

```
class MyClass:
```

```
    x = 5
```

Create Object

Now we can use the class named MyClass to create objects:

Example:

Create an object named p1, and print the value of x:

```
p1 = MyClass()
```

```
print(p1.x)
```

Delete Objects

You can delete objects by using the **del** keyword:

Example:

Delete the p1 object:

```
del p1
```

Multiple Objects

You can create multiple objects from the same class

Example:

Create three objects from the MyClass class:

```
p1 = MyClass()
```

```
p2 = MyClass()
```

```
p3 = MyClass()
```

```
print(p1.x)
```

```
print(p2.x)
```

```
print(p3.x)
```

Note: Each object is independent and has its own copy of the class properties.

The pass Statement

class definitions cannot be empty, but if you for some reason have a **class** definition with no content, put in the **pass** statement to avoid getting an error.

Example:

```
class Person:
    pass
```

The __init__() Method

All classes have a built-in method called `__init__()`, which is always executed when the class is being initiated.

The `__init__()` method is used to assign values to object properties, or to perform operations that are necessary when the object is being created.

Example:

Create a class named Person, use the `__init__()` method to assign values for name and age:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil", 36)
```

```
print(p1.name)
print(p1.age)
```

Note: The `__init__()` method is called automatically every time the class is being used to create a new object.

Why Use __init__()?

Without the `__init__()` method, you would need to set properties manually for each object:

Example:

Create a class without `__init__()`:

```
class Person:
    pass
```

```
p1 = Person()
p1.name = "Tobias"
p1.age = 25
```

```
print(p1.name)
print(p1.age)
```

Using `__init__()` makes it easier to create objects with initial values:

Example

With `__init__()`, you can set initial values when creating the object:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Linus", 28)
```

```
print(p1.name)
print(p1.age)
```

Default Values in `__init__()`

You can also set default values for parameters in the `__init__()` method:

Example:

Set a default value for the age parameter:

```
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil")
p2 = Person("Tobias", 25)
```

```
print(p1.name, p1.age)
print(p2.name, p2.age)
```

Multiple Parameters

The `__init__()` method can have as many parameters as you need:

Example:

Create a Person class with multiple parameters:

```
class Person:
    def __init__(self, name, age, city, country):
        self.name = name
        self.age = age
        self.city = city
        self.country = country

p1 = Person("Linus", 30, "Oslo", "Norway")

print(p1.name)
print(p1.age)
print(p1.city)
print(p1.country)
```

The `self` Parameter

The `self` parameter is a reference to the current instance of the class.

It is used to access properties and methods that belong to the class.

Example:

Use `self` to access class properties:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print("Hello, my name is " + self.name)

p1 = Person("Emil", 25)
p1.greet()
```

Note: The `self` parameter must be the first parameter of any method in the class.

Why Use self?

Without `self`, Python would not know which object's properties you want to access:

Example:

The `self` parameter links the method to the specific object:

```
class Person:
    def __init__(self, name):
        self.name = name

    def printname(self):
        print(self.name)

p1 = Person("Tobias")
p2 = Person("Linus")

p1.printname()
p2.printname()
```

self Does Not Have to Be Named "self"

It does not have to be named `self`, you can call it whatever you like, but it has to be the **first parameter** of any method in the class:

Example:

Use the words *myobject* and *abc* instead of *self*:

```
class Person:
    def __init__(myobject, name, age):
        myobject.name = name
        myobject.age = age

    def greet(abc):
        print("Hello, my name is " + abc.name)

p1 = Person("Emil", 36)
p1.greet()
```

Note: While you *can* use a different name, it is strongly recommended to use `self` as it is the convention in Python and makes your code more readable to others.

Accessing Properties with self

You can access any property of the class using `self`:

Example:

Access multiple properties using `self`:

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year

    def display_info(self):
        print(f"{self.year} {self.brand} {self.model}")

car1 = Car("Toyota", "Corolla", 2020)
car1.display_info()
```

Calling Methods with self

You can also call other methods within the class using `self`:

Example:

Call one method from another method using `self`:

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return "Hello, " + self.name

    def welcome(self):
        message = self.greet()
        print(message + "! Welcome to our website.")

p1 = Person("Tobias")
p1.welcome()
```

Python Class Properties

Class Properties

Properties are variables that belong to a class. They store data for each object created from the class.

Example:

Create a class with properties:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Emil", 36)

print(p1.name)
print(p1.age)
```

Access Properties

You can access object properties using dot notation:

Example:

Access the properties of an object:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

car1 = Car("Toyota", "Corolla")

print(car1.brand)
print(car1.model)
```


Modify Properties

You can modify the value of properties on objects:

Example:

Change the age property:

```
Ac class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Tobias", 25)
print(p1.age)

p1.age = 26
print(p1.age)
```

Delete Properties

You can delete properties from objects using the [del](#) keyword:

Example:

Delete the age property:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Linus", 30)

del p1.age

print(p1.name) # This works
# print(p1.age) # This would cause an error
```

Class Properties vs Object Properties

Properties defined inside `__init__()` belong to each object (instance properties).

Properties defined outside methods belong to the class itself (class properties) and are shared by all objects:

Example:

Class property vs instance property:

```
class Person:
    species = "Human" # Class property

    def __init__(self, name):
        self.name = name # Instance property

p1 = Person("Emil")
p2 = Person("Tobias")

print(p1.name)
print(p2.name)
print(p1.species)
print(p2.species)
```

Modifying Class Properties

When you modify a class property, it affects all objects:

Example:

Change the class property:

```
class Person:
    lastname = ""

    def __init__(self, name):
        self.name = name

p1 = Person("Linus")
p2 = Person("Emil")

Person.lastname = "Refsnes"

print(p1.lastname)
print(p2.lastname)
```

Add New Properties

You can add new properties to existing objects:

Example:

Add a new property to an object:

```
class Person:
    def __init__(self, name):
        self.name = name

p1 = Person("Tobias")

p1.age = 25
p1.city = "Oslo"

print(p1.name)
print(p1.age)
print(p1.city)
```

Note: Adding properties this way only adds them to that specific object, not to all objects of the class.

Python Class Methods

Class Methods

Methods are functions that belong to a class. They define the behaviour of objects created from the class.

Example:

Create a method in a class:

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello, my name is " + self.name)

p1 = Person("Emil")
p1.greet()
```

Note: All methods must have `self` as the first parameter.

Methods with Parameters

Methods can accept parameters just like regular functions:

Example:

Create a method with parameters:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b

calc = Calculator()
print(calc.add(5, 3))
print(calc.multiply(4, 7))
```

Methods Accessing Properties

Methods can access and modify object properties using `self`:

Example:

A method that accesses object properties:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def get_info(self):
        return f"{self.name} is {self.age} years old"

p1 = Person("Tobias", 28)
print(p1.get_info())
```

Methods Modifying Properties

Methods can modify the properties of an object:

Example:

A method that changes a property value:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def celebrate_birthday(self):
        self.age += 1
        print(f"Happy birthday! You are now {self.age}")

p1 = Person("Linus", 25)
p1.celebrate_birthday()
p1.celebrate_birthday()
```

The `__str__()` Method

The `__str__()` method is a special method that controls what is returned when the object is printed:

Example:

Without the `__str__()` method:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("Emil", 36)
print(p1)
```

Example:

With the `__str__()` method:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} ({self.age})"
```

```
p1 = Person("Tobias", 36)
print(p1)
```

Multiple Methods

A class can have multiple methods that work together:

Example:

Create multiple methods in a class:

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.songs = []

    def add_song(self, song):
        self.songs.append(song)
        print(f"Added: {song}")

    def remove_song(self, song):
        if song in self.songs:
            self.songs.remove(song)
            print(f"Removed: {song}")

    def show_songs(self):
        print(f"Playlist '{self.name}':")
        for song in self.songs:
            print(f"- {song}")

my_playlist = Playlist("Favorites")
my_playlist.add_song("Bohemian Rhapsody")
my_playlist.add_song("Stairway to Heaven")
my_playlist.show_songs()
```

Delete Methods

You can delete methods from a class using the `del` keyword:

Example:

Delete a method from a class:

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello!")

p1 = Person("Emil")

del Person.greet

p1.greet() # This will cause an error
```